

time or risking failure. Claude Code also struggled to remain focused on a single attack path. While attempting to determine the ActiveMQ version, it would abruptly switch to trying the default-credential brute force. After selecting and spending a long time configuring a Metasploit module, it might suddenly divert to investigating another script found on Exploit-DB, creating needless context switches and time loss. Finally, because Claude Code lacked explicit, structured reasoning, it failed to map the discovered ActiveMQ version to the most appropriate CVE. As a result, it missed the more effective Metasploit module and wasted excessive time on two suboptimal exploits.

### E. Ablation Study

In this section, we conduct an ablation study by comparing CHECKMATE with two commonly used strategies for enhancing LLM-based systems. First, we compare CHECKMATE against the RAG-based approach, which is an alternative strategy for expanding an LLM’s knowledge base. We embedded metadata of specialized penetration tools, including more than 14 thousand Metasploit modules, NSE scripts, and Nuclei templates, as the document database and implemented a RAG pipeline. We aim to evaluate whether LLM agents can effectively use external knowledge to improve their penetration capabilities without relying on predefined actions and classical planning+. Second, we let Claude Code maintain a structured planning file in JSON format rather than using its default to-do list. The prompting was modified so that after each command execution, Claude Code updates this structured planning file and infers the next step based on the revised state. This approach reflects common methodologies in prior work that employ structured planning representations to improve an LLM agent’s planning consistency. For each method, we evaluated the performance on 20 tasks, running each task three times. All four methods successfully obtained a remote shell at least once. As shown in Figure 7, CHECKMATE achieves the lowest overall cost and the shortest execution time, while also delivering the most consistent and efficient performance across test cases. These results indicate that although incorporating RAG or structured planning files can enhance the efficiency of LLM-based agents, the classical planning+ approach provides the most substantial gains in both efficiency and consistency.

## VI. DISCUSSION AND FUTURE WORK

### A. Actions and States in Pentesting

Existing work leaves two fundamental questions unanswered: (1) What actions and skills does pentesting require? and (2) How should we represent the state of the target system? The difficulty arises from the open-ended nature of pentesting. Unlike tasks with well-defined action and state spaces, pentesting spans the full breadth of a system’s architecture, configurations, vulnerabilities, and defenses, and demands a wide and adaptable skill set. Current approaches either define fixed, finite sets of skills and states, or depend heavily on black-box LLMs to infer target states and propose actions. The fixed schemas are too restrictive, while relying on opaque LLMs makes it difficult to systematically improve penetration

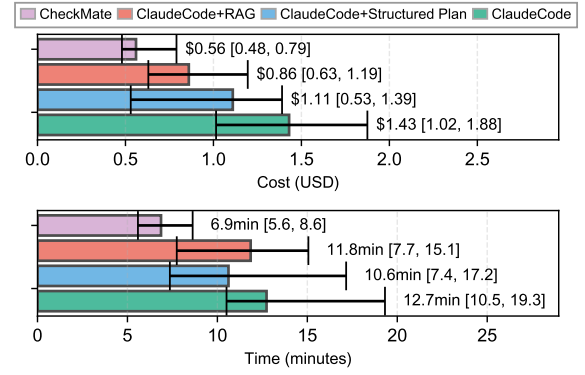


Fig. 7: Cost and time comparison. (a) Median API costs in USD. (b) Median execution time in minutes. Error bars represent the interquartile range (25th-75th percentile).

capabilities. This gap highlights the need for future work on representing, organizing, extracting, and operationalizing the fragmented knowledge on actions and states in pentesting.

### B. Multimodal and UI-Aware Pentesting

Existing pentesting systems struggle in scenarios that require rich human-computer interaction, as traditional LLM agents are not good at interpreting non-textual information and operating web user interfaces (UIs) like a human. As a result, tasks that involve understanding visual elements or manipulating dynamic, interactive web components still depend heavily on humans. Recent advances in multimodal learning and Customizable UI Automation (CUA) offer promising avenues for addressing these limitations [36], [37], opening up new possibilities for pentesting in complex UI environments.

## VII. CONCLUSION

In this paper, we first presented a systematic review of existing automated pentesting work through the lens of our Planner-Executor-Perceptor (PEP) paradigm. Our evaluation shows that the out-of-the-box Claude Code+Sonnet 4.5 substantially outperforms all prior systems in this area. However, further analysis revealed three limitations of Claude Code. We thus proposed CHECKMATE, a framework that couples classical planning+ with LLM agents to address these limitations. Experimental evaluations demonstrated that CHECKMATE outperforms existing systems in penetration capability, efficiency, and stability.

## ETHICAL CONSIDERATIONS

This paper presents a practical study on using LLM Agents for pentesting. All techniques and systems involved are publicly accessible; we have not developed any new zero-day attacks. All experiments were conducted within authorized virtual environments. We will contact the service providers to inform them of the potential for their products in offensive scenarios. This work is intended solely for research and educational purposes, and we do not encourage or endorse any misuse of the discussed techniques.

## REFERENCES

- [1] Cybersecurity and Infrastructure Security Agency, “Penetration testing services,” <https://www.cisa.gov/resources-tools/services/penetration-testing>, 2023, u.S. Department of Homeland Security.
- [2] MarketsandMarkets, “Penetration testing market size, size, growth & latest trends,” <https://www.marketsandmarkets.com/Market-Reports/penetration-testing-market-13422019.html>, 2024, accessed: 2025-10-03.
- [3] —, “Penetration testing as a service market size & share analysis – global forecast to 2029,” <https://www.marketsandmarkets.com/Market-Reports/penetration-testing-as-a-service-market-36245315.html>, 2024, accessed: 2025-10-03.
- [4] V. Mayoral-Vilches, L. J. Navarrete-Lozano, M. Sanz-Gómez, L. S. Espejo, M. Crespo-Álvarez, F. Oca-Gonzalez, F. Balassone, A. Glerapicón, U. Ayucar-Carbajo, J. A. Ruiz-Alcalde, S. Rass, M. Pinzger, and E. Gil-Uriarte, “Cai: An open, bug bounty-ready cybersecurity ai,” 2025. [Online]. Available: <https://arxiv.org/abs/2504.06017>
- [5] B. Wu, G. Chen, K. Chen, X. Shang, J. Han, Y. He, W. Zhang, and N. Yu, “Autopt: How far are we from the endend automated web penetration testing?” *arXiv preprint arXiv:2411.01236*, 2024.
- [6] C. Sarraute, O. Buffet, and J. Hoffmann, “Pomdps make better hackers: Accounting for uncertainty in penetration testing,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 26, no. 1, 2012, pp. 1816–1824.
- [7] J. Hoffmann, “Simulated penetration testing: from” dijkstra” to” turing test++,” in *Proceedings of the international conference on automated planning and scheduling*, vol. 25, 2015, pp. 364–372.
- [8] I. Mirzadeh, K. Alizadeh, M. Shahrokhi, O. Tuzel, S. Bengio, and M. Farajtabar, “Gsm-symbolic: Understanding the limitations of mathematical reasoning in large language models,” *arXiv preprint arXiv:2410.05229*, 2024.
- [9] B. Y. Lin, R. L. Bras, K. Richardson, A. Sabharwal, R. Poovendran, P. Clark, and Y. Choi, “Zeballogic: On the scaling limits of llms for logical reasoning,” *arXiv preprint arXiv:2502.01100*, 2025.
- [10] K. Yamin, S. Gupta, G. R. Ghosal, Z. C. Lipton, and B. Wilder, “Failure modes of llms for causal reasoning on narratives,” *arXiv preprint arXiv:2410.23884*, 2024.
- [11] H. Chi, H. Li, W. Yang, F. Liu, L. Lan, X. Ren, T. Liu, and B. Han, “Unveiling causal reasoning in large language models: Reality or mirage?” *Advances in Neural Information Processing Systems*, vol. 37, pp. 96 640–96 670, 2024.
- [12] J. Liu, K. Wang, Y. Chen, X. Peng, Z. Chen, L. Zhang, and Y. Lou, “Large language model-based agents for software engineering: A survey,” *arXiv preprint arXiv:2409.02977*, 2024.
- [13] H. Jin, L. Huang, H. Cai, J. Yan, B. Li, and H. Chen, “From llms to llm-based agents for software engineering: A survey of current, challenges and future,” *arXiv preprint arXiv:2408.02479*, 2024.
- [14] Y. Wang, W. Zhong, Y. Huang, E. Shi, M. Yang, J. Chen, H. Li, Y. Ma, Q. Wang, and Z. Zheng, “Agents in software engineering: Survey, landscape, and vision,” *Automated Software Engineering*, vol. 32, no. 2, pp. 1–36, 2025.
- [15] S. Ullah, M. Han, S. Pujar, H. Pearce, A. Coskun, and G. Stringhini, “Llms cannot reliably identify and reason about security vulnerabilities (yet?): A comprehensive evaluation, framework, and benchmarks,” in *2024 IEEE symposium on security and privacy (SP)*. IEEE, 2024, pp. 862–880.
- [16] J. Guo, C. Wang, X. Xu, Z. Su, and X. Zhang, “Repoaudit: An autonomous llm-agent for repository-level code auditing,” *arXiv preprint arXiv:2501.18160*, 2025.
- [17] M. Rahman, A. Bhuiyan, M. S. Islam, M. T. R. Laskar, R. Mahbub, A. Masry, S. Joty, and E. Hoque, “Llm-based data science agents: A survey of capabilities, challenges, and future directions,” *arXiv preprint arXiv:2510.04023*, 2025.
- [18] Anthropic, “Claude code,” <https://www.claude.com/product/claude-code>, 2025, accessed: 2025-11-07.
- [19] M. Ghallab, D. Nau, and P. Traverso, *Automated Planning: theory and practice*. Elsevier, 2004.
- [20] “Vulhub: Open-source vulnerable docker environments,” <https://vulhub.org/>, accessed: 2025-12-04.
- [21] J. Schwartz, H. Kurniawati, and E. El-Mahassni, “Pomdp+ information-decay: Incorporating defender’s behaviour in autonomous penetration testing,” in *Proceedings of the International Conference on Automated Planning and Scheduling*, vol. 30, 2020, pp. 235–243.
- [22] C. Sarraute, O. Buffet, and J. Hoffmann, “Penetration testing== pomdp solving?” *arXiv preprint arXiv:1306.4714*, 2013.
- [23] J. Schwartz and H. Kurniawati, “Autonomous penetration testing using reinforcement learning,” *arXiv preprint arXiv:1905.05965*, 2019.
- [24] M. C. Ghanem, T. M. Chen, and E. G. Nepomuceno, “Hierarchical reinforcement learning for efficient and effective automated penetration testing of large networks,” *Journal of Intelligent Information Systems*, vol. 60, no. 2, pp. 281–303, 2023.
- [25] S. Zhou, J. Liu, D. Hou, X. Zhong, and Y. Zhang, “Autonomous penetration testing based on improved deep q-network,” *Applied Sciences*, vol. 11, no. 19, p. 8823, 2021.
- [26] G. D. Pasquale, I. Grishchenko, R. Iesari, G. Pizarro, L. Cavallaro, C. Kruegel, and G. Vigna, “ChainReactor: Automated privilege escalation chain discovery via AI planning,” in *33rd USENIX Security Symposium (USENIX Security 24)*. Philadelphia, PA: USENIX Association, Aug. 2024, pp. 5913–5929. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity24/presentation/de-pasquale>
- [27] J. L. Obes, C. Sarraute, and G. Richarte, “Attack planning in the real world,” *arXiv preprint arXiv:1306.4044*, 2013.
- [28] Z. Chen, F. Kang, X. Xiong, and H. Shu, “A survey on penetration path planning in automated penetration testing,” *Applied Sciences*, vol. 14, no. 18, p. 8355, 2024.
- [29] Z. Wang, Y. Zhang, Z. Liu, X. Wei, Y. Chen, and B. Wang, “An automatic planning-based attack path discovery approach from it to ot networks,” *Security and Communication Networks*, vol. 2021, no. 1, p. 1444182, 2021.
- [30] G. Deng, Y. Liu, V. Mayoral-Vilches, P. Liu, Y. Li, Y. Xu, T. Zhang, Y. Liu, M. Pinzger, and S. Rass, “{PentestGPT}: Evaluating and harnessing large language models for automated penetration testing,” in *33rd USENIX Security Symposium (USENIX Security 24)*, 2024, pp. 847–864.
- [31] Armur-Ai, “Auto-pentest-gpt-ai: Llm powered pentesting for your software,” <https://github.com/Armur-Ai/Auto-Pentest-GPT-AI>, 2025, accessed: 2025-10-16.
- [32] J. Huang and Q. Zhu, “Penheal: A two-stage llm framework for automated pentesting and optimal remediation,” in *Proceedings of the workshop on autonomous cybersecurity*, 2023, pp. 11–22.
- [33] J. Xu, J. W. Stokes, G. McDonald, X. Bai, D. Marshall, S. Wang, A. Swaminathan, and Z. Li, “Autoattacker: A large language model guided system to implement automatic cyber-attacks,” *arXiv preprint arXiv:2403.01038*, 2024.
- [34] H. Kong, D. Hu, J. Ge, L. Li, T. Li, and B. Wu, “Vulnbot: Autonomous penetration testing for a multi-agent collaborative framework,” *arXiv preprint arXiv:2501.13411*, 2025.
- [35] GH05TCREW, “Pentestagent: All-in-one offensive security toolbox with ai agent and mcp architecture,” <https://github.com/GH05TCREW/PentestAgent>, 2025, accessed: 2025-10-16.
- [36] X. Wang, B. Wang, D. Lu, J. Yang, T. Xie, J. Wang, J. Deng, X. Guo, Y. Xu, C. H. Wu *et al.*, “Opencua: Open foundations for computer-use agents,” *arXiv preprint arXiv:2508.09123*, 2025.
- [37] Y. Yang, D. Li, Y. Dai, Y. Yang, Z. Luo, Z. Zhao, Z. Hu, J. Huang, A. Saha, Z. Chen *et al.*, “Gta1: Gui test-time scaling agent,” *arXiv preprint arXiv:2507.05791*, 2025.
- [38] A. L. Blum and M. L. Furst, “Fast planning through planning graph analysis,” *Artificial intelligence*, vol. 90, no. 1-2, pp. 281–300, 1997.
- [39] S. Kambhampati, K. Valmeekam, L. Guan, M. Verma, K. Stechly, S. Bhambri, L. Saldyt, and A. Murthy, “Llms can’t plan, but can help planning in llm-modulo frameworks,” *arXiv preprint arXiv:2402.01817*, 2024.
- [40] P. Cao, T. Men, W. Liu, J. Zhang, X. Li, X. Lin, D. Sui, Y. Cao, K. Liu, and J. Zhao, “Large language models for planning: A comprehensive and systematic survey,” *arXiv preprint arXiv:2505.19683*, 2025.
- [41] H. Zhang, H. Deng, J. Ou, and C. Feng, “Mitigating spatial hallucination in large language models for path planning via prompt engineering,” *Scientific Reports*, vol. 15, no. 1, p. 8881, 2025.
- [42] Z. Ji, D. Wu, P. Ma, Z. Li, and S. Wang, “Testing and understanding erroneous planning in llm agents through synthesized user inputs,” *arXiv preprint arXiv:2404.17833*, 2024.
- [43] Z. Yao, Y. Liu, Y. Chen, J. Chen, J. Fang, L. Hou, J. Li, and T.-S. Chua, “Are reasoning models more prone to hallucination?” *arXiv preprint arXiv:2505.23646*, 2025.

- [44] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, “Lost in the middle: How language models use long contexts,” *Transactions of the Association for Computational Linguistics*, vol. 12, pp. 157–173, 2024.
- [45] X. Shen, L. Wang, Z. Li, Y. Chen, W. Zhao, D. Sun, J. Wang, and W. Ruan, “Pentestagent: Incorporating llm agents to automated penetration testing,” in *Proceedings of the 20th ACM Asia Conference on Computer and Communications Security*, 2025, pp. 375–391.
- [46] “picoCTF,” <https://picoctf.org/>, accessed: 2025-12-04.
- [47] “Hack The Box,” <https://www.hackthebox.com/>, accessed: 2025-12-04.
- [48] M. Shao, S. Jancheska, M. Udeshi, B. Dolan-Gavitt, K. Milner, B. Chen, M. Yin, S. Garg, P. Krishnamurthy, F. Khorrami *et al.*, “Nyu ctf bench: A scalable open-source benchmark dataset for evaluating llms in offensive security,” *Advances in Neural Information Processing Systems*, vol. 37, pp. 57 472–57 498, 2024.
- [49] Y. Ginige, A. Niroshan, S. Jain, and S. Seneviratne, “Autopentester: An llm agent-based framework for automated pentesting,” *arXiv preprint arXiv:2510.05605*, 2025.
- [50] “Xbow: Ai-powered penetration testing platform,” <https://xbow.com/>, XBOW USA Inc., 2025, accessed: 2025-10-16.
- [51] 0x4m4, “Hexstrike ai mcp agents,” <https://github.com/0x4m4/hexstrike-ai>, 2025, accessed: 2025-10-16.
- [52] OpenAI, “Openai codex,” <https://openai.com/codex/>, 2025, accessed: 2025-11-07.
- [53] Google, “Code assist,” <https://codeassist.google/>, 2025, accessed: 2025-11-07.